Assignment - 14

1) What do you mean by class variable & instance variable of class.

→ Static characteristics of a class = Class variable
Non-static characteristics of a class = Instance variable

1) Class variable :
- It is associated with class rather than individual object
- Memory for static characteristics variables gets allocated by default & only once
- Should be initialized outside class using 'Scope Resolution operator (::)'
- Can access w/o creating the object with help of class name & scope resolution operator.

2) Instance variable of class
- Belongs to a specific object (instance) of the class
- memory of non-static chars gets allocated separately for each object
- Accessed using a specific object of class
- Used to store data unique to each instance of a class

* Code → * static2.cpp

2) What is meant by argument? Explain Default Argument
→ There are 2 types of arguments in function
1) Regular / Compulsory arguments
2) Default Argument * default.cpp

- Type of argument which is considered as a optional
- If we skip that parameter while calling the func, then its default value gets considered
- In above program, Calculate(), takes 2 parameters ← marks (Compulsory) → must be passed
out of (optional)

* If we skip 'out of' parameter then its default value is considered as 100.

Rules:

All default arguments must be at the last in func^n argument list otherwise, compiler will generate an error.

E.g.: float calculate (float out = 100, float marks)

If we call this func^n, with only 1 variable, the compiler will get confused, as in to which parameter the value should be assigned.

3] Difference bet^n Static & Non-static Characteristics of class

	Static	Non-static
Associat^n	Belongs to class itself	Belongs to individual objects (instances)
Storage	Allocated once, shared across all objects.	Separate storage for each object.
Initialization	Done outside the class for static data	Done per object, typically through constructors.
Access	Accessed using the class name or object	Accessed only via an object
Member functions	Cannot access non-static members	Can access both static & non-static members.
Use case	Shared or global state utility functions	Instance-specific data & behaviour.

4] Explain Parameterized Constructor with Default Arguments.

→ We can use concept of default arguments in case of constructor.

If we create a parametrized constructor which uses the default arguments in it then that type of constructor is called → parameterized Constructor with arguments.

* Static1.cpp

In above program we can't write :
a) Default constructor.
b) Parametrized constructor with 1 argument
c) ——— 1 ——— 2 arguments.

5] What concept of Name Mangling?

→ 1) When we compile the code, the compiler changes the name of every function with "mangled name" (modified name)
2) When we overload the function, all the names of function are same but due to Name Mangling the compiler will change name of every function with new name as per pattern.
3) No concept of function overloading after code gets compiled!

Name mangling

int Addition (int no1, int no2)

Addition @2ii = Addition @ 2 i i
initials of data types of every parameter.

* Refer → overloading1.cpp

6] How do we initialize the static characteristics of a class?

→ - Static characteristics should be initialized outside class using 'scope resolution operator (::)'.
- Declare the static data member using the 'static' keyword

Eg: static int var; // declare
int myclass::var = 0; // defn & initialize

7] Can we access private non-static characteristics of a class from static method & Explain with example.

→ - No, we cannot access private non-static chars of a class directly, as static methods are not tied to any specific instance of the class.
- You can access pvt non-static chars indirectly using a reference/pointer to an instance of class passed to the static method.
- Code:

```
#include <iostream>
using namespace std;
Class MyClass {
private:
    int nonStaticVar;
    Static int staticVar;
public:
    MyClass (int value) : nonStaticVar (value) { }

    Static void display ()
    { cout << " Static Variable: " << staticVar << endl; }
};
```

```
int MyClass :: staticVar = 10;
int main()
{ MyClass obj(20);
  MyClass :: display();
  return 0;
}
```

Error output → error: invalid use of non-static data member 'nonStaticVar'

8] Is it possible to create private static characteristics of class? Explain with example.

→ - Yes, it is possible to create private static characteristics of class in C++
- These are defined with 'private Access Specifier' & are only accessible within the class
- They are not accessible directly from outside the class, even using the class name or an object.
- Static class declared as private can only be accessed or modified through public member functions, including static member functions.

```
#include <iostream>
using namespace std;

Class MyClass
{ private:
    static int privateStaticVar; // Pvt static member
  public:
    Static void setPrivateStaticVar (int value)
    { privateStaticVar = value; }

    Static int getPrivateStaticVar ()
    { return privateStaticVar; }
```

```
};  // Initialize pvt static var outside class
int MyClass :: privateStaticVar = 0;

int main()
{  // Accessing through public static member functions
   MyClass :: setPrivateStaticVar(42);
   cout << "Private Static Variable: " << MyClass ::
   getPrivateStaticVar() << endl;
   return 0;
}
```

Output → Private Static Variable : 42

9] What is the lifetime (scope) of static characteristics of class?

→ - They are created (allocated) only once when the program starts.
- They persist throughout the entire lifetime of the program, regardless of how many objects of the class are created or destroyed.
- They are destroyed (deallocated) when the program ends.
- Scope is determined by Access Specifier.

10] What is meant by static Behaviour? Explain with example.

→ // Static3.cpp
- The above Thali class contains below things:
1) Non-static Characteristics (variable): Jamun & Rasgulla.
2) Static Characteristics : Loncha
3) Non-static behaviour : void display()
4) Static Behaviour : static void showLoncha()

- In the above app^n 'Showloancha' is a static method.
- To call this static method we can use the name of that class.
- Static method can be called w/o creating object.
- Static method can access only static characteristics of a class.
∴ - Non-static can access (static + Non-static) characteristics
- We can call static method by using the object but the compiler internally converts it into name of class.
- All the above points above → same in Java.